

Bölüm 1 — Temeller (Chapter 1)

Programlama Dilleri

Kavramları ve Tasarımı

Tüm Sorular — İngilizce & Türkçe + Cevaplar

Kaynak Kitap:

Concepts of Programming Languages

Robert W. Sebesta — 12. Baskı

29

Gözden Geçirme
Sorusu

18

Problem
Seti Sorusu

47

Toplam
Soru

İkiDil

EN + TR
İçerik

■ Gözden Geçirme Soruları (Review Questions)

■ Problem Seti (Problem Set)

BÖLÜM 1 — GÖZDEN GEÇİRME SORULARI (Review Questions) - 29

Soru

Soru 1

EN: Why is it useful for a programmer to have some background in language design, even though he or she may never actually design a programming language?

TR: Bir programcının dil tasarımı konusunda neden bilgi sahibi olması faydalıdır?

● Cevap

Bir programcı hiçbir zaman yeni bir dil tasarlamayacak olsa bile, dil tasarım ilkelerini anlamak ona mevcut dilleri daha etkin kullanma imkânı sunar. Hangi yapıların neden var olduğunu bilen bir programcı dili daha bilinçli kullanır, hataları daha iyi anlar ve yeni dillere çok daha hızlı adapte olur.

Soru 2

EN: How can knowledge of programming language characteristics benefit the whole computing community?

TR: Programlama dili özelliklerine dair bilgi tüm bilgisayar topluluğuna nasıl fayda sağlar?

● Cevap

Dil özellikleri hakkında geniş bir bilgi birikimi, daha iyi araçların (derleyici, hata ayıklayıcı, IDE) geliştirilmesine katkıda bulunur. Yazılım kalitesini artırır; daha okunabilir, bakımı kolay ve güvenilir sistemlerin üretilmesine zemin hazırlar. Sonuç olarak tüm kullanıcılar daha güvenilir yazılımlardan yararlanır.

Soru 3

EN: What programming language has dominated scientific computing over the past 60 years?

TR: Son 60 yılda bilimsel hesaplama hâkim olan programlama dili hangisidir?

● Cevap

Fortran (Formula Translation), 1957'de IBM tarafından geliştirilmiş olup son 60 yılı aşkın sürede bilimsel hesaplama alanında baskın dil olmuştur. Yüksek performanslı sayısal hesaplama, matris işlemleri ve fizik/mühendislik simülasyonlarında bugün hâlâ yaygın biçimde kullanılmaktadır.

Soru 4

EN: What programming language has dominated business applications over the past 60 years?

TR: Son 60 yılda iş uygulamalarına hâkim olan programlama dili hangisidir?

● Cevap

COBOL (Common Business-Oriented Language), 1959'da tasarlanmış olup iş dünyası uygulamalarında (bankacılık, muhasebe, stok yönetimi vb.) son 60 yılın hâkim dili olmuştur. Büyük kuruluşların eski (legacy) sistemlerinde hâlâ milyarlarca satır COBOL kodu aktif olarak çalışmaktadır.

Soru 5

EN: What programming language has dominated artificial intelligence over the past 60 years?

TR: Son 60 yılda yapay zekâya hâkim olan programlama dili hangisidir?

● Cevap

LISP (List Processing), 1958'de John McCarthy tarafından geliştirilen bu fonksiyonel dil; sembolik işlem, liste manipülasyonu ve özyineleme yetenekleri sayesinde yapay zekâ araştırmalarında onlarca yıl boyunca baskın dil olmuştur. Günümüzde Common Lisp ve Scheme gibi türevleri hâlâ kullanılmaktadır.

Soru 6

EN: In what language is most of UNIX written?

TR: UNIX işletim sisteminin büyük bölümü hangi dille yazılmıştır?

● Cevap

UNIX büyük ölçüde C programlama diliyle yazılmıştır. 1970'lerin başında Dennis Ritchie ve Ken Thompson tarafından Bell Labs'da geliştirilen C dili, işletim sistemi programlamasında taşınabilirlik ve verimlilik sağlamış; o tarihten bu yana sistem yazılımlarının temel dili hâline gelmiştir.

Soru 7

EN: What is the disadvantage of having too many features in a language?

TR: Bir dilde çok fazla özellik bulunmasının dezavantajı nedir?

● Cevap

Bir dilde gereğinden fazla özellik bulunması, o dili öğrenmeyi ve kullanmayı güçleştirir. Farklı programcılar aynı problemi farklı özelliklerle çözebilir; bu da okunabilirliği azaltır ve ekip çalışmalarında tutarsızlığa yol açar. Ayrıca derleyici karmaşıklığı artar ve dilin bakımı daha maliyetli hâle gelir.

Soru 8

EN: How can user-defined operator overloading harm the readability of a program?

TR: Kullanıcı tanımlı operatör aşırı yükleme (operator overloading) bir programın okunabilirliğini nasıl olumsuz etkiler?

● Cevap

Operatör aşırı yükleme (operator overloading) hakkı kötüye kullanıldığında, bir operatörün gerçek anlamı bağlama göre tamamen farklılaşabilir. Örneğin '+' operatörü bir sınıfta liste birleştirme, başkasında vektör toplama için tanımlanabilir. Kodu okuyan kişi her sınıfın tanımını incelemek zorunda kalır; bu da okunabilirliği ciddi ölçüde azaltır.

Soru 9

EN: What is one example of a lack of orthogonality in the design of C?

TR: C dilinin tasarımında ortogonallik (orthogonality) eksikliğine bir örnek veriniz.

● Cevap

C dilinde struct (yapı) nesneleri fonksiyonlara değer olarak (pass by value) geçirilebilirken, diziler (array) bu şekilde geçirilemez ve her zaman işaretçi (pointer) olarak iletilir. Bu tutarsızlık, benzer veri türlerinin farklı kurallara tabi olduğunu gösterir.

Soru 10

EN: What language used orthogonality as a primary design criterion?

TR: Ortogonallığı birincil tasarım kriteri olarak benimseyen dil hangisidir?

● Cevap

ALGOL 68, ortogonallığı temel tasarım ilkesi olarak benimsemiş bir dildir. Bu dilde az sayıda temel yapı ve veri türü, mümkün olan her kombinasyonda tutarlı biçimde bir araya getirilebilir. Ancak bu ilkenin aşırı uygulanması dilin son derece karmaşık görünmesine yol açmıştır.

Soru 11

EN: What primitive control statement is used to build more complicated control statements in languages that lack them?

TR: Gelişmiş kontrol ifadelerinden yoksun dillerde karmaşık kontrol yapıları hangi temel deyimle inşa edilir?

● Cevap

goto (koşulsuz dal/atlama) deyimi, döngü ve koşullu yapı gibi üst düzey kontrol mekanizmalarına sahip olmayan dillerde karmaşık kontrol akışı oluşturmak için kullanılır. Ancak goto'nun kontrolsüz kullanımı 'spagetti kodu' olarak adlandırılan okunaksız kod yapısına yol açar.

Soru 12

EN: What does it mean for a program to be reliable?

TR: Bir programın güvenilir (reliable) olması ne anlama gelir?

● Cevap

Güvenilir bir program, tüm koşullar altında — normal ve anormal girdiler dahil — beklenen şekilde davranır. Güvenilirlik; tür denetimi (type checking), özel durum işleme (exception handling), değişken kapsamı kontrolü ve tutarlı okuma/yazma işlemleri gibi bir dizi özelliği kapsar.

Soru 13

EN: Why is type checking the parameters of a subprogram important?

TR: Bir alt programın (subprogram) parametrelerinin tür denetimden (type checking) geçirilmesi neden önemlidir?

● Cevap

Parametre tür denetimi, bir alt programa yanlış türde argüman gönderilmesini engeller. Bu sayede tür uyumsuzluğundan kaynaklanan çalışma zamanı hataları (runtime error) derleme aşamasında yakalanabilir ve programın güvenilirliği artar.

Soru 14

EN: What is aliasing?

TR: Takma ad (aliasing) nedir?

● Cevap

Takma ad (aliasing), bellekteki aynı konuma birden fazla farklı isimle erişilebilmesi durumudur. Örneğin iki işaretçi (pointer) aynı bellek hücrelerini gösteriyorsa, birindeki değişiklik diğerini de etkiler. Bu durum programın anlaşılmasını güçleştirir ve optimizasyon yapan derleyicilerin işini karmaşıktırır.

Soru 15

EN: What is exception handling?

TR: İstisna işleme (exception handling) nedir?

● Cevap

İstisna işleme (exception handling), programın normal akışını bozan anormal durumların (sıfıra bölme, dosya bulunamama, bellek yetersizliği vb.) yakalanmasını ve önceden tanımlanmış biçimlerde ele alınmasını sağlayan mekanizmadır. Ada, C++, Java ve Python bu mekanizmayı try-catch-finally yapılarıyla destekler.

Soru 16

EN: Why is readability important to writability?

TR: Okunabilirlik (readability) neden yazılabilirliği (writability) etkiler?

● Cevap

Okunabilir bir dil, programcının kendi yazdığı kodu ya da başkasının kodunu kolayca anlayabilmesine olanak tanır. Kodun anlaşılır olması mantık hatalarının daha çabuk fark edilmesini sağlar. Programcı başkasının kodunu anlayabildiğinde, o kodu model alarak yeni kod yazmak çok daha kolaylaşır.

Soru 17

EN: How is the cost of compilers for a given language related to the design of that language?

TR: Bir dil için derleyici (compiler) maliyeti ile dilin tasarımı arasındaki ilişki nedir?

● Cevap

Bir dilin karmaşık ve tutarsız yapısı, derleyicinin daha büyük, karmaşık ve geliştirilmesi daha maliyetli olmasına yol açar. Tasarımı basit ve ortogonal olan diller için derleyici yazmak görece kolaydır; bu da birden fazla platform için derleyici üretimine ve daha az hatalı implementasyonlara olanak tanır.

Soru 18

EN: What have been the strongest influences on programming language design over the past 60 years?

TR: Son 60 yılda programlama dili tasarımı üzerindeki en güçlü etkiler neler olmuştur?

● Cevap

Programlama dili tasarımı etkileyen başlıca unsurlar: (1) Bilgisayar mimarisindeki gelişmeler (von Neumann mimarisi), (2) Yazılım geliştirme metodolojilerindeki ilerlemeler (yapısal, nesne yönelimli, fonksiyonel programlama), (3) Uygulama alanlarının genişlemesi (bilimsel hesaplama, web, gömülü sistemler), (4) Derleyici teknolojisindeki ilerlemeler.

Soru 19

EN: What is the name of the category of programming languages whose structure is dictated by the von Neumann computer architecture?

TR: Yapısı von Neumann bilgisayar mimarisinden türetilen programlama dili kategorisinin adı nedir?

● Cevap

Bu kategori imperatif diller (imperative languages) ya da işlemsel diller (procedural languages) olarak adlandırılır. von Neumann mimarisine dayanan bu dillerde program, bellekte depolanan değişkenler üzerinde sıralı komutlar aracılığıyla işlem yapar. C, Pascal ve Fortran bu grubun tipik örnekleridir.

Soru 20

EN: What two programming language deficiencies were discovered as a result of the research in software development in the 1970s?

TR: 1970'lerdeki yazılım geliştirme araştırmaları sonucunda hangi iki programlama dili yetersizliği keşfedilmiştir?

● Cevap

1970'lerdeki yazılım krizi (software crisis) araştırmaları iki temel yetersizliği ortaya koymuştur: (1) goto deyiminin kontrolsüz kullanımı nedeniyle oluşan karmaşık, bakımı imkânsız kod yapısı — yapısal programlama ile çözülmeye çalışıldı; (2) Veri ve işlemlerin birbirinden ayrı tutulması, verinin korunaksız biçimde erişilebilir olması — nesne yönelimli programlama ile ele alındı.

Soru 21

EN: What are the three fundamental features of an object-oriented programming language?

TR: Nesne yönelimli programlama dilinin üç temel özelliği nelerdir?

● Cevap

Nesne yönelimli programlamanın (object-oriented programming) üç temel özelliği: (1) Kapsülleme (encapsulation) — veri ve işlemlerin nesnelerde bir arada barındırılması, (2) Kalıtım (inheritance) — mevcut sınıflardan yeni sınıflar türetilmesi, (3) Çok biçimlilik (polymorphism) — aynı arayüzün farklı nesne türleri için farklı biçimlerde gerçekleştirilmesi.

Soru 22

EN: What language was the first to support the three fundamental features of object-oriented programming?

TR: Nesne yönelimli programlamanın üç temel özelliğini destekleyen ilk dil hangisidir?

● Cevap

Simula 67 (1967), nesne yönelimli programlamanın üç temel özelliğini — kapsülleme, kalıtım ve çok biçimliliği — ilk kez bir arada destekleyen dildir. Norveçli bilgisayar bilimciler Ole-Johan Dahl ve Kristen Nygaard tarafından geliştirilen Simula 67, sonraki tüm nesne yönelimli dillere ilham kaynağı olmuştur.

Soru 23

EN: What is an example of two language design criteria that are in direct conflict with each other?

TR: Doğrudan çatışan iki dil tasarım kriterine örnek veriniz.

● Cevap

Güvenilirlik (reliability) ile verimlilik (efficiency) sıkça çatışan iki kriterdir. Örneğin C dilinde dizi sınırı denetimi (array bounds checking) varsayılan olarak yapılmaz; bu durum performansı artırır ancak güvenilirliği düşürür. Benzer şekilde esneklik (flexibility) ile okunabilirlik (readability) de çatışabilir.

Soru 24

EN: What are the three general methods of implementing a programming language?

TR: Bir programlama dilini uygulamanın (implement etmenin) üç genel yöntemi nelerdir?

● Cevap

Üç temel uygulama yöntemi: (1) Derleme (compilation) — kaynak kod makine koduna çevrilerek doğrudan çalıştırılır (C, C++); (2) Saf yorumlama (pure interpretation) — kaynak kod çalışma zamanında satır satır yorumlanır; (3) Hibrit uygulama (hybrid implementation) — kaynak kod önce bytecode'a dönüştürülür, ardından sanal makinede yorumlanır (Java, Python).

Soru 25

EN: Which produces faster program execution, a compiler or a pure interpreter?

TR: Program yürütmeyi daha hızlı gerçekleştiren hangisidir: derleyici mi yoksa saf yorumlayıcı mı?

● Cevap

Derleyici (compiler), kaynak kodu doğrudan makine koduna çevirdiğinden çalışma zamanı performansı çok daha yüksektir. Saf yorumlayıcı (pure interpreter) her deyimi çalışma zamanında çözümlemek zorunda olduğundan genellikle derlenmiş koda kıyasla 10–100 kat daha yavaş çalışır.

Soru 26

EN: What role does the symbol table play in a compiler?

TR: Bir derleyicide (compiler) sembol tablosunun (symbol table) rolü nedir?

● Cevap

Sembol tablosu (symbol table), derleme sürecinde tanımlanan tüm tanımlayıcıları (değişken, fonksiyon, tür adı vb.) adları, türleri, kapsamları ve bellek adresleriyle birlikte saklayan bir veri yapısıdır. Derleyici bu tabloyu tür denetimi yapmak, kapsam kurallarını uygulamak ve kod üretimi sırasında tanımlayıcılara başvurmak için kullanır.

Soru 27

EN: What does a linker do?

TR: Bağlayıcı (linker) ne iş yapar?

● Cevap

Bağlayıcı (linker), ayrı ayrı derlenen nesne dosyalarını (.o / .obj) ve kütüphane dosyalarını bir araya getirerek tek bir yürütülebilir program oluşturur. Modüller arası fonksiyon çağrılarını ve değişken referanslarını çözümler; bellek adreslerini düzenler ve programın çalıştırılmaya hazır hâle gelmesini sağlar.

Soru 28

EN: Why is the von Neumann bottleneck important?

TR: von Neumann darboğazı (von Neumann bottleneck) neden önemlidir?

● Cevap

von Neumann mimarisinde işlemci (CPU) ile bellek arasındaki veri yolu (bus) aynı anda yalnızca sınırlı miktarda veri taşıyabilir. Program ve veri aynı bellekte bulunduğundan, her komut için bellekten hem komutun hem de verilerin okunması gerekir. Bu darboğaz, işlemci ile bellek hızları arasındaki büyüyen uçurum nedeniyle günümüzde de temel bir performans sorunu olmaya devam etmektedir.

Soru 29

EN: What are the advantages in implementing a language with a pure interpreter?

TR: Saf yorumlayıcı (pure interpreter) kullanarak bir dili uygulamanın avantajları nelerdir?

● Cevap

Saf yorumlayıcının başlıca avantajları: (1) Hata iletileri daha açıklayıcıdır; yorumlayıcı hatanın tam satırını gösterebilir, (2) Hata ayıklama (debugging) daha kolaydır; çalışırken değişken değerleri incelenebilir, (3) Platform bağımsızlığı sağlar; yorumlayıcı olan her sistemde çalışır, (4) Geliştirme-test döngüsü daha hızlıdır; derleme adımı gerektirmez.

BÖLÜM 1 — PROBLEM SETİ (Problem Set) · 18 Soru**Problem 1**

EN: Do you believe our capacity for abstract thought is influenced by our language skills? Support your opinion.

TR: Soyut düşünme kapasitemizin dil becerilerimizden etkilendiğine inanıyor musunuz? Görüşünüzü destekleyin.

● Cevap

Evet, dil ve düşünce birbirini derinden etkiler — Sapir-Whorf hipotezi bu ilişkiyi inceler. Bir programlama dilinde rekürsif düşünmeyi öğrenen birinin soyut problem çözme becerisinin geliştiği gözlemlenir. Kelime dağarcığı ve dil yapıları, kavramsal sınırları belirler; bu durum hem doğal diller hem de programlama dilleri için geçerlidir.

Problem 2

EN: What are some features of specific programming languages you know whose rationales are a mystery to you?

TR: Bildiğiniz programlama dillerinde gerekçesini anlamakta güçlük çektiğiniz özellikler var mı?

● Cevap

JavaScript'te == operatörünün tür dönüştürme (type coercion) yapması (örn. '5' == 5 sonucunun true olması), C++'ta çoklu kalıtımın (multiple inheritance) karmaşık elmas problemi (diamond problem) yaratması ve Python'da girintinin (indentation) sözdizimsel anlam taşıması bu tür gizemli özellikler arasında sayılabilir.

Problem 3

EN: What arguments can you make for the idea of a single language for all programming domains?

TR: Tüm programlama alanları için tek bir dilin kullanılması fikrinin lehinde ne gibi argümanlar öne sürebilirsiniz?

● Cevap

Tek bir dilin avantajları: (1) Programcılar arasında iletişim ve iş birliği kolaylaşır, (2) Kütüphane ve araç ekosistemi tek noktada toplanır, (3) Eğitim maliyetleri düşer, (4) Kod yeniden kullanımı (code reuse) artar, (5) Platformlar arası standartlaşma sağlanır. Python'un pek çok alanda yaygınlaşması bu yaklaşımın potansiyelini göstermektedir.

Problem 4

EN: What arguments can you make against the idea of a single language for all programming domains?

TR: Tüm programlama alanları için tek bir dilin kullanılması fikrinin aleyhinde ne gibi argümanlar öne sürebilirsiniz?

● Cevap

Tek bir dilin dezavantajları: (1) Farklı alanların farklı gereksinimleri vardır; sistem programlaması verimlilik isterken, yapay zekâ ifade gücü ister, (2) Tek bir dil tüm alanlarda ortalamada kalır, hiçbirinde mükemmel olmaz, (3) Rekabet ve çeşitlilik yenilikçiliği teşvik eder, (4) Özelleşmiş diller alan uzmanlığı geliştirmeye yardımcı olur.

Problem 5

EN: Name and explain another criterion by which languages can be judged (in addition to those discussed in this chapter).

TR: Bu bölümde tartışılanların yanı sıra dilleri değerlendirmek için kullanılabilecek başka bir kriter belirtin ve açıklayın.

● Cevap

Taşınabilirlik (portability): Bir dilde yazılmış programın farklı donanım ve işletim sistemlerinde değişiklik gerektirmeksizin çalışabilmesi. Java'nın 'Write Once, Run Anywhere' felsefesi bu kriterin en iyi örneğidir. Taşınabilirlik; geliştirme maliyetini düşürür ve yazılımın ömrünü uzatır.

Problem 6

EN: What common programming language statement, in your opinion, is most detrimental to readability?

TR: Sizce hangi yaygın programlama dili deyimi okunabilirliğe en fazla zarar vermektedir?

● Cevap

goto deyimi okunabilirliğe en fazla zarar veren yapıdır. Programın akışını öngörülmez biçimlerde değiştiren goto, 'spagetti kodu' (spaghetti code) olarak bilinen doğrusal olmayan, takibi neredeyse imkânsız kod yapısına yol açar. Edsger Dijkstra'nın 1968'deki ünlü 'Go To Statement Considered Harmful' makalesi bu tehlikeyi ayrıntılı biçimde ortaya koymuştur.

Problem 7

EN: Java uses a right brace to mark the end of all compound statements. What are the arguments for and against this design?

TR: Java tüm bileşik deyimlerin sonunu sağ parantez (}) ile işaretler. Bu tasarımın lehinde ve aleyhinde ne gibi argümanlar vardır?

● Cevap

Lehinde: Tutarlı ve evrensel bir sözdizimi; IDE'lerde otomatik girintilemeyi kolaylaştırır; okuyucu her zaman kapanışın nereden yapıldığını bilir. Aleyhinde: İç içe yapılarda çok sayıda '}' art arda gelerek okunabilirliği bozabilir; Pascal'daki end gibi anahtar kelimeler hangi bloğun kapandığını daha açık ifade edebilir.

Problem 8

EN: Many languages distinguish between uppercase and lowercase letters in user-defined names. What are the pros and cons of this design decision?

TR: Pek çok dil büyük/küçük harf ayrımı (case sensitivity) yapar. Bu tasarım kararının artıları ve eksileri nelerdir?

● Cevap

Artıları: Daha fazla anlamlı isim kullanılabilir (örn. Node ve node farklı şeyler olabilir); sözleşmeye dayalı yazım kuralları anlam katabilir. Eksileri: Yazım hatalarını bulmak güçleşir (örn. Counter ve counter); dil öğrenimini zorlaştırır; büyük/küçük harf hatalarından kaynaklanan sinsi hatalar (subtle bugs) oluşabilir.

Problem 9

EN: Explain the different aspects of the cost of a programming language.

TR: Bir programlama dilinin maliyetinin farklı boyutlarını açıklayın.

● Cevap

Programlama dili maliyeti birçok boyutu kapsar: (1) Eğitim maliyeti — programcıları yetiştirme süresi ve bedeli, (2) Yazım maliyeti — program geliştirme hızı, (3) Derleme maliyeti — derleyicinin satın alma/geliştirme gideri, (4) Yürütme maliyeti — çalışma zamanı performansı, (5) Bakım maliyeti — uzun vadeli destek ve hata düzeltme, (6) Derleyici/araç geliştirme maliyeti.

Problem 10

EN: What are the arguments for writing efficient programs even though hardware is relatively inexpensive?

TR: Donanımın görece ucuz olmasına rağmen verimli programlar yazmanın lehinde ne gibi argümanlar vardır?

● Cevap

(1) Büyük ölçekli sistemlerde küçük verimsizlikler büyük kayıplara dönüşür; (2) Mobil ve gömülü sistemlerde enerji tüketimi kritiktir; (3) Verimli algoritmalar ölçeklenebilirliği (scalability) artırır; (4) Kullanıcı deneyimi (UX) yavaş yazılımdan doğrudan etkilenir; (5) Bulut maliyetleri verimsiz kodla katlanarak artar.

Problem 11

EN: Describe some design trade-offs between efficiency and safety in some language you know.

TR: Bildiğiniz bir dilde verimlilik ve güvenlik arasındaki tasarım ödünleşimlerini (trade-offs) açıklayın.

● Cevap

C dilinde: Dizi sınır denetimi (array bounds checking) yoktur — bu verimlilik sağlar ama tampon taşması (buffer overflow) güvenlik açıklarına yol açar. Bellek yönetimi programcıya bırakılmıştır — hız kazanılır ama bellek sızıntısı (memory leak) ve sarkan işaretçi (dangling pointer) riskleri doğar. Java bu dengeyi otomatik bellek yönetimi ve dizi denetimiyle güvenlik lehine kurar; ancak performans bedelini öder.

Problem 12

EN: In your opinion, what major features would a perfect programming language include?

TR: Sizce mükemmel bir programlama dili hangi temel özellikleri içermelidir?

● Cevap

Mükemmel bir dil şunları içermelidir: (1) Güçlü tür sistemi (strong type system) — güvenilirlik için; (2) Otomatik bellek yönetimi — güvenlik için; (3) Sade ve tutarlı sözdizimi (syntax) — okunabilirlik için; (4) Yüksek performans — verimlilik için; (5) Çoklu paradigma desteği (imperatif, fonksiyonel, nesne yönelimli); (6) Zengin standart kütüphane; (7) Güçlü eşzamanlılık (concurrency) desteği.

Problem 13

EN: Was the first high-level programming language you learned implemented with a pure interpreter, a hybrid implementation system, or a compiler?

TR: Öğrendiğiniz ilk üst düzey programlama dilinin uygulaması nasıldı: saf yorumlayıcı mı, hibrit sistem mi yoksa derleyici mi?

● Cevap

Bu soru kişisel deneyime göre değişmekle birlikte, yaygın bir örnek vermek gerekirse: Python hibrit uygulama (hybrid implementation) kullanan bir dildir. Python kaynak kodu önce bytecode'a (.pyc) derlenir, ardından CPython sanal makinesi tarafından yorumlanır. Java da benzer biçimde JVM (Java Virtual Machine) üzerinde çalışan bytecode üretir.

Problem 14

EN: Describe the advantages and disadvantages of some programming environment you have used.

TR: Kullandığınız bir programlama ortamının avantajlarını ve dezavantajlarını açıklayın.

● Cevap

Visual Studio Code örneği: Avantajlar — hafif ve hızlı başlatma, zengin eklenti ekosistemi, Git entegrasyonu, çok dil desteği, ücretsiz ve açık kaynak. Dezavantajlar — büyük projelerde bellek tüketimi artabilir, bazı diller için IntelliSense (kod tamamlama) eksik kalabilir, tam IDE'lerin sunduğu gelişmiş refactoring araçlarından yoksun olabilir.

Problem 15

EN: How do type declaration statements for simple variables affect the readability of a language, considering that some languages do not require them?

TR: Bazı dillerde tür bildirimi (type declaration) zorunlu değilken, basit değişkenler için tür bildirimi yapılmasının okunabilirliğe etkisi nedir?

● Cevap

Açık tür bildirimleri (explicit type declarations), değişkenin ne tür veri tuttuğunu belgeleme işlevi görür ve kodun anlaşılmasını kolaylaştırır. Öte yandan tür çıkarımı (type inference) kullanan dillerde (Kotlin, Swift, Haskell) kod daha kısa ve az tekrarlı olabilir. Ancak bildirimsiz dillerde büyük kod tabanlarında değişken türünü tahmin etmek güçleşebilir.

Problem 16

EN: Write an evaluation of some programming language you know, using the criteria described in this chapter.

TR: Bu bölümde açıklanan kriterleri kullanarak bildiğiniz bir programlama dilini değerlendirin.

● Cevap

Python değerlendirmesi: Okunabilirlik — çok yüksek (sade sözdizimi, anlamlı girintileme). Yazılabilirlik — yüksek (kısa ve ifade gücü yüksek). Güvenilirlik — orta (dinamik tiplendirme hata riskini artırır). Verimlilik — düşük-orta (yorumlanan dil; NumPy gibi C uzantıları telafi eder). Taşınabilirlik — yüksek (çapraz platform). Genel değerlendirme: Prototiplendirme ve veri bilimi için mükemmel; sistem programlaması için yetersiz.

Problem 17

EN: Some programming languages—for example, Pascal—have used the semicolon to separate statements, while Java uses it to terminate statements. Which of these, in your opinion, is most natural and least likely to result in syntax errors?

TR: Pascal noktalı virgüllü (;) deyimleri ayırmak için, Java ise sonlandırmak için kullanır. Sizce hangisi daha doğal ve daha az sözdizim hatasına yol açar?

● Cevap

Noktalı virgülü sonlandırıcı (terminator) olarak kullanan Java yaklaşımı daha az sözdizim hatasına yol açar. Her deyim kendi kapanışını taşıması, deyim nereden bittiğini kesin biçimde belirtir. Ayırıcı (separator) kullanan Pascal'da, son deyimden sonra gereksiz ';' koymak ya da bir önceki deyimden sonra ';' unutmak yaygın hatalardır.

Problem 18

EN: Many contemporary languages allow two kinds of comments: one in which delimiters are used on both ends (multiple-line comments), and one in which a delimiter marks only the beginning of the comment (one-line comments). Discuss the advantages and disadvantages of each.

TR: Pek çok modern dil iki tür yorum (comment) satırına izin verir: çok satırlı (/* ... */) ve tek satırlı (//). Her birinin avantajlarını ve dezavantajlarını tartışın.

● Cevap

Çok satırlı yorumlar (/* ... */): Avantaj — uzun açıklamalar ve kod bloklarını geçici olarak devre dışı bırakmak için idealdir. Dezavantaj — iç içe yorumlara (nested comments) izin verilmediğinde kapanış işaretini unutmak büyük bölümleri hatalı biçimde yorum satırına dönüştürebilir. Tek satırlı yorumlar (//): Avantaj — satır sonu otomatik kapanış sağlar, iç içe kullanım sorunları yoktur. Dezavantaj — çok satırlı açıklamalar için her satıra // eklemek gerekir.